

Figure 2.9 After formatting your data, you'll need to measure it to ensure that the graphics you create are appropriately sized and positioned based on the parameters of the dataset.

Now that we've loaded our data and transformed it into types that are accessible, we'll investigate the patterns of that data by measuring the data (the third step shown in figure 2.9).

2.1.4 Measuring data

After loading your data array, one of the first things you should do is measure and sort it. It's particularly important to know the distribution of values of particular attributes, as well as the minimum and maximum values and the names of the attributes. D3 provides a set of array functions that can help you understand your data.

You'll always have arrays filled with data that you'll want to size and position based on the relative value of an attribute compared to the distribution of the values in the array. You should therefore familiarize yourself with the ways to determine the distributions of values in an array in D3. You'll work with an array of numbers first before you see these functions in operation with more complex and more data-rich JSON object arrays:

```
var testArray =  [88,10000,1,75,12,35];
```

Nearly all the D3 measuring functions follow the same pattern. First, you need to designate the array and an accessor function for the value that you want to measure. In our case, we're working with an array of numbers and not an array of objects, so the accessor only needs to point at the element itself.

```
d3.min(testArray, function (el) {return el});
d3.max(testArray, function (el) {return el});
d3.mean(testArray, function (el) {return el});
```

Returns the minimum value in the array, 1

Returns the maximum value in the array, 10000

Returns the average of values in the array, 1701.8333333333335

If you're dealing with a more complex JSON object array, then you'll need to designate the attribute you want to measure. For instance, if we're working with the array of JSON objects from cities.csv, we may want to derive the minimum, maximum, and average populations:

Returns the minimum value of the population attribute of each object in the array, 500000

Returns the maximum value of the population attribute of each object in the array, 1300000

```
d3.csv("cities.csv", function(data) {
d3.min(data, function (el) {return +el.population});
d3.max(data, function (el) {return +el.population });
```